

Supplementary Material for “RAI: Safe Pipelined Epoch Transitions for Account-Local Payments”

Anonymous Author(s)

Regular paper (supplement, not part of the page budget)

This supplement complements the submission. Appendix A gives the full proofs of the results of Section VI and the counterexample that motivates the early-vote rule. Appendix A describes the three TLA⁺ models, their abstractions, configurations, and results. Appendix A lists the scripted regression cases derived from the models and their status in the prototype. Appendix A records the prototype’s departures from the protocol, the defects found while measuring, and two additional experiments. Numbering of theorems and lemmas follows the main text; section, algorithm, and table references are to the main text. Throughout, $N = 3f + 2p + 1$, $q = N - f - p$, $r = f + p + 1$, and reports are selected in sets of $N - f$.

APPENDIX

Lemma 1 (Account exclusion): In one account instance at most one block has an exclusion witness; in particular at most one has an NC. If an FC or FF for B can be formed in that instance, no conflicting block has an exclusion witness. If an FF for B can be formed, every conflicting block has at most $f + p < r$ supporters.

Proof: Two sets of q supporters intersect in at least $2q - N = f + 1$ identities, one of them correct, and a correct validator supports one block per instance (Definition 1), by a first vote or by a late notarization. An FC for B has q final voters; a correct final voter first-voted B in the same instance, so the same intersection applies against any XW of a rival. An FF has $N - p \geq q$ first voters. For the last claim let $f' \leq f$ be the actual number of Byzantine members. An FF has at least $N - p - f'$ correct first voters, so at most p correct members and f' Byzantine members can support a rival: at most $p + f' \leq p + f$. ■

Lemma 2 (Late notarization adds no old finality): Every epoch- e NC, FC, or FF consists of first and final votes released by correct signers before they left epoch e , together with Byzantine votes. A correct validator releases a final vote, or records an N entry, only for a block for which it holds an NC of q first votes.

Proof: By Definition 1 and the certificate definitions of Section IV-B none of the three certificates counts a late notarization, and a correct validator releases no epoch- e first or final vote after leaving e (Section V-A). Completeness is defined on NC _{e} , and notarization and final voting require completeness.

Lemma 3 (Frozen-report exposure): Let i be a correct reporter for closure e . If i released an epoch- e final vote for B , then T_i tags B with N or F. If i released an epoch- e first vote for B , then T_i tags B with N or F, or $h(B) \in G_i$. This holds whatever other tag B inherits from S_{e-1} , and it persists after the report is frozen.

Proof: A final vote follows durable notarization, which is an N entry unless finality has replaced it by F. For a first vote, $h(B) \in V_i^{(e)}$, and G_i removes from $V_i^{(e)}$ exactly the hashes tagged N or F. Stopping precedes freezing; roots and tags are then immutable, and C1 ensures that what a correct validator reconstructs is what was frozen. ■

Remark 1: With the earlier definition $G_i = V_i^{(e)} \setminus \text{keys}(T_i)$ the second statement fails when S_{e-1} carries B under a recovery record: T_i tags B with R and the fresh vote appears nowhere. `RaiClose` with `FIXB = FALSE` reaches two conflicting fast certificates this way (Appendix A).

Lemma 4 (Signers hold the witness): Every correct signer of an eligible epoch- e FC or FF for B holds, as retained evidence, an exclusion witness discharging each recovery record of S_{e-1} on a block that conflicts with B .

Proof: An FF _{e} (B) consists of settled first votes. A correct signer of one had installed S_{e-1} when signing; the voting rule of Section V-F refuses a rival of a retained block unless every record on it is discharged, and the signer’s admission witness fixes the witnesses it used, so it holds them. A correct final voter either released its final vote after installing S_{e-1} , with the same check, or before. In the second case the early-vote rule required it to hold XW _{$e-1$} (B) for B and every unresolved block of the selected prefix. That witness discharges every record on a rival that closure $e - 1$ created, since those have origin $e - 1$ (Rule 2 stamps the closing epoch). Records that S_{e-1} inherits from S_{e-2} were known to the voter, because S_{e-2} is installed before the overlap of closure $e - 1$ begins (at most one closing epoch per validator), and the overlap guard required the matching-origin witness for each of them. ■

Theorem 1 (Closure preserves latent and late finality): Let S_{e-1} be a valid closed predecessor that satisfies the closure invariant, and let v be any candidate with $\text{Eligible}_e(v)$ and state S_e . Let B have an eligible epoch- e account final certificate that can be formed, possibly completed after reports froze or after the checkpoint was decided, and assume

■(W) B conflicts with no lock record carried by S_{e-1} , or every

correct signer of the certificate holds, as retained evidence, an exclusion witness that discharges each such record.

Under the early-vote rule, (W) holds for every eligible certificate (Lemma 4); without that rule it can fail (Appendix A). Then S_e holds B as final, under a notarization lock, or under a recovery record of origin e , directly or as an ancestor of a block so held; no admissible evidence discharges that hold; and S_e satisfies the closure invariant for all certificates covered by (W).

Proof: Let Q be the selected reporters, $|Q| = N - f$, and let A be the signer set of the certificate.

Eligibility of B in the construction. If B conflicts with no carried record there is nothing to show. Otherwise, by (W), each correct signer holds an exclusion witness for every carried record that B bypasses. That witness is retained evidence; reconstructing the signer's report material delivers it, the manifest contains it, and the record is discharged at line 6 of Algorithm 1. So B is eligible for Rules 1 and 2 whenever a correct signer is selected, which the counts below guarantee.

Normal certificate. A is a set of q final voters, so $|(Q \cap A) \setminus \text{Byz}| \geq (N - f) + q - N - f = q - 2f = p + 1 \geq 1$. By Lemma 3 each such reporter froze an N or F entry for B . A verified F entry puts B in F^* . Otherwise B is a represented NC target; it is the unique one at its position by Lemma 1, it does not conflict with F^* by the induction hypothesis of Theorem 2, and Rule 1 retains it under a notarization lock.

Fast certificate. A is a set of $N - p$ settled first voters, so $|(Q \cap A) \setminus \text{Byz}| \geq (N - f) + (N - p) - N - f = N - p - 2f = r$. Take these reporters. If any selected report has a verified F entry for B we are in the first case. Else if any has a verified N entry for B , Rule 1 applies as above. Else, by Lemma 3, each of the r reporters has $h(B) \in G_i$ with a verified first vote, whether or not T_i tags B with R; hence $|U_Q(B)| \geq r$. No rival has a represented NC, and every rival B' has $|U_Q(B')| \leq f + p < r$, both by Lemma 1. Rule 2 therefore adds a recovery record of origin e for B .

No discharge. A notarization lock is never bypassed. A record of origin e on B is discharged only by $XW_e(B')$ for a conflicting B' , which Lemma 1 excludes because an epoch- e final certificate for B can be formed. Evidence of other epochs does not apply to it, and discharging records of other origins on B leaves it in place.

Earlier epochs. For $e' < e$ the predecessor holds B as final, under a notarization lock, or under a record of origin e' . Finality is inherited. A notarization lock and a record of origin e' are carried forward by the inherited-state rule, since the only ways to drop them are a conflicting final block, excluded by Theorem 2, and an $XW_{e'}$ of a rival, excluded by Lemma 1.

The argument uses only signatures in A and their intersection with Q , so it does not matter when the certificate is assembled, and unselected or Byzantine signatures cannot remove the correct intersections. ■

Lemma 5 (Checkpoint-promotion safety): A Rule 3 promoted block cannot conflict with a block that has a closing-epoch exclusion witness, FC, or FF. Hence a decision that

finalizes the promoted prefix is compatible with old-domain finality and with core overlap finality.

Proof: Every promoted block has a verified closing-epoch NC of first votes, which is an exclusion witness; Lemma 1 excludes an exclusion witness, and so an FC or FF, for a conflicting block. Core overlap finality for a conflicting successor block requires a closing-epoch exclusion witness at the first divergent position, which is the same contradiction. Promotion stops at retained forks and never promotes recovery-only evidence. ■

Lemma 6 (Acyclic final dependencies): For any finite set of valid final blocks closed under account ancestors and required finalized sends, parent and send-to-receive edges form a directed acyclic graph.

Proof: Order blocks by the earliest event at which valid final evidence justifies them. Along a parent edge this event cannot become earlier at the child; along a send-to-receive edge it is strictly later at the receive, because admission required the send's finality proof and Rule 3 cannot supply it for the checkpoint under construction. A cycle cannot consist only of parent edges, since slots increase strictly, so it contains a send-to-receive edge and a strict increase around a cycle, a contradiction. ■

Lemma 7 (The overlap verdict does not depend on installation order): Let closure e be in progress, let B and B' conflict at one position, and let OC be an overlap certificate for B : an exclusion witness $XW_e(B)$, an $NC_{e+1}(B)$, an epoch- $(e + 1)$ FC or FF for B or a descendant selecting it, and an exclusion witness of the matching origin for every recovery record on B' that S_{e-1} carries, with no notarization lock on B' in S_{e-1} . Then for every candidate with Eligible $_e$ and state S_e : (a) S_e does not finalize B' ; (b) S_e carries no notarization lock on B' ; (c) every recovery record that S_e carries on B' is discharged by a witness contained in OC. Consequently a validator that holds OC accepts B as final under the overlap exception before installing S_e if and only if it accepts B by ordinary rechecking after installing S_e , and a later manifest that contains OC verifies an F claim for B against S_e .

Proof: (a) S_e finalizes a block through inherited finality, an explicit closing-epoch account certificate, or Rule 3. Inherited finality of B' contradicts the check of OC against S_{e-1} , which is installed before the overlap of closure e begins. An epoch- e FC or FF for B' , and the $NC_e(B')$ that Rule 3 requires, are excluded by $XW_e(B)$ and Lemma 1. (b) A new notarization lock needs a represented $NC_e(B')$, excluded the same way; an inherited one is excluded by hypothesis. (c) A record on B' in S_e is either created by Rule 2 at this closure, with origin e , and then $XW_e(B) \subseteq \text{OC}$ discharges it; or inherited from S_{e-1} , and then OC contains its discharge witness by hypothesis.

For the consequence: before installation the validator checks exactly the contents of OC against S_{e-1} . After installation, ordinary rechecking tests B against the frontier, the finalized history, and the carried records of S_e ; by (a)–(c) these tests pass on the evidence in OC and on no other. The same evaluation is what line 6 of Algorithm 1 performs at closure

$e + 1$. The lemma is stated for one divergent position; the prefix-wide guard applies it at the first divergent unresolved position of the finalized prefix, as the proof of Theorem 2 does. ■

Lemma 8 (Deterministic construction): Assume the predecessor satisfies the closure invariant. For fixed usable selected reports and a valid immutable manifest, BuildState returns one canonical candidate state at every correct validator. It preserves inherited finality, carries all mandatory surviving records, and applies only explicit account finality plus Rule 3 promotion. All-correct usable inputs yield a finite valid construction.

Proof: By C1 every correct validator reconstructs the same (T_i, G_i) for each selected header, and the manifest fixes witnesses and proof choices; hence explicit finality, discharges, and R_Q are the same everywhere, with lock records compared as sets of (block, strength, origin). Lemma 1, predecessor validity, attachment, and Theorem 1 make the retained finality and records compatible. Rule 3's tests fix P_Q , safe by Lemma 5; Lemma 6 and the canonical tie-break fix replay. An N entry of a correct reporter is backed by an NC of first votes (Lemma 2), which the manifest can carry, so the entry verifies. An F entry of a correct reporter for a block finalized under the overlap exception is backed by its overlap certificate, which is retained evidence; by Lemma 7 line 6 accepts the claim and discharges every conflicting record of S_{e-1} , so no all-correct selection is rejected on that account. Malformed Byzantine inputs are excluded by usability; all-correct usable inputs yield a finite state. ■

Theorem 2 (Transfer safety across epochs): Assuming C1 and A1–A3, finalized histories at correct validators are compatible and irreversible across all core-protocol epochs; finalized balances are nonnegative, and each send is received at most once.

Proof: Induct from genesis. Within an epoch, Lemma 1 excludes incompatible account finality. At a boundary, A2 ensures that the decided v_e is eligible, so by Theorem 1 it preserves every formable old final branch and by Lemma 5 its promotion is compatible; A1 and A3 ensure that every correct validator installs the state of the same v_e , which by Lemma 8 is the same state. For overlap, the first divergent unresolved position on an early successor-final prefix has a closing-epoch exclusion witness for the successor block by the prefix-wide guard, whose late notarizations add no old finality that Theorem 1 would have to preserve (Lemma 2); a conflicting closing-epoch certificate or a conflicting Rule 3 promotion would require a conflicting NC, contradicting Lemma 1; and by Lemma 7 correct validators that hold the same overlap certificate reach the same verdict whether they verify it before or after installing S_e , so they agree on which overlap blocks are final and not only on the compatibility of what they have finalized. Matching-origin discharge and inherited records remain enforced after installation, and the base/overlay union with applied-operation identifiers prevents rollback and duplicate application. Replay checks balances and finalized-send dependencies. ■

Lemma 9 (Correct reports become usable): Under the data

service, finite relevant admitted work, fair processing, and C2, every correct report eventually becomes usable at every correct validator.

Proof: A correct reporter stops old first and final voting, so its old vote set and roots are finite, and its report is well formed: every N entry rests on an NC of first votes (Lemma 2), every F entry on inherited finality, an account certificate, or a retained overlap certificate, and every G_i member on its own first vote and on no hash tagged N or F. By C2 every correct validator eventually reconstructs (T_i, G_i) . The data service delivers the blocks, ancestry, signed votes, late notarizations, and overlap certificates behind its claims, and fair processing completes the finite validation. A withholding Byzantine reporter can make only its own report unusable; $N - f$ correct reports remain. ■

Theorem 3 (Handoff termination): Assume the data service, fair processing, C2, A4, and recurring prepared windows with $N - f$ responsive correct members of K_e long enough for reconstruction, candidate preparation, and the consensus service's rounds. Every enabled handoff is eventually decided and, with eventual retained-data delivery, installed by correct successors.

Proof: By Lemma 9, $N - f$ correct reports become usable at every correct member, which by Lemma 8 can then construct an eligible candidate; the canonical selection order makes correct members that hold the same usable reports construct the same one. A4 then decides some eligible value in a sufficient window; A3 lets successors verify it, and retained authorizing data let them recompute and install the state, which enables the next deferred boundary. ■

Theorem 4 (Uncontested account continuation): Assume handoff termination, available final-send dependencies, and recurring common-open ready intervals. An owner following the retry-or-fresh-child policy of Section IV eventually finalizes each target of a finite valid uncontested continuation, provided each required new position has such an interval, and the finalized targets eventually enter a checkpoint.

Proof: In a ready interval, $N - f \geq q$ correct first votes create an NC and the same validators can final-vote; after they install the predecessor their first votes are settled, so an available $N - p$ fast quorum finalizes in two stages. An attempt interrupted by a boundary is preserved by Theorem 1; an omitted block can be retried, a uniquely retained notarized target may finalize at the handoff, and a remaining retained fork can be selected by a fresh uncontested child. Repeating this over the finite continuation and using eventual evidence delivery gives finalization and checkpoint inclusion. ■

A. Early votes against a carried lock

Without the early-vote rule, hypothesis (W) of Theorem 1 is not cosmetic. The following run violates the theorem in the protocol as it stood before the repairs, and equally after Fix B alone. RaiHandoff finds it with `RULE = FALSE`. Take $N = 6$ with validator 6 Byzantine, and conflicting blocks B, B' .

- 1) Epoch e : validators 1, 2, 6 first-vote B' . Validators 3, 4, 5 cross their boundary without voting at this position.
- 2) Before S_e is known, 3, 4, 5 first-vote B in epoch $e + 1$ and late-notarize it in e . Validator 6 signs both votes for B and shows its late notarization to validator 1 only.
- 3) Closure e selects reports 1, 2, 3, 4, 6. Rule 2 gives B' a recovery record of origin e .
- 4) Validators 3, 4, 5 install S_e without holding $XW_e(B)$; their early work on B is invalid in their view. Validator 1 installs S_e , holds $XW_e(B) = \{3, 4, 5, 6\}$, discharges the record and first-votes B in $e + 1$. Now $\{1, 3, 4, 5, 6\}$ is an $FF_{e+1}(B)$ that can be formed.
- 5) Closure $e + 1$ selects reports 2, 3, 4, 5, 6. The only correct holder of the witness is not selected, the manifest has no $XW_e(B)$, the votes for B are an unexplained speculative path, and S_{e+1} retains B' only.
- 6) In $e + 2$ validators 2, 3, 4, 5, 6 first-vote the retained B' : $FF_{e+2}(B')$. Validator 6 then publishes its signatures; B has an FF_{e+1} whose eligibility against S_e follows from $XW_e(B)$.

With a committee change the same failure needs no late notarization: four new members early-vote B , the old committee's first votes for B plus one withheld Byzantine vote are the exclusion witness, and the second closure sees four unexplained votes. The cause is that the eligibility of an early first vote can depend on a witness its signer never held. The early-vote rule removes the dependency: in step 4 the set $\{1, 3, 4, 5, 6\}$ is no longer a fast certificate, because three of its votes are early, and validators 3, 4, 5 final-vote B only once they hold $XW_e(B)$, in which case their reports deliver it to the next closure.

Two alternatives were considered and set aside. Retaining both blocks at the second closure keeps the one-round path but can stall a contested position permanently after a committee change. Counting only early votes of members of the closing committee is unsafe: the committee-change run above defeats it.

The three modules, their TLC configurations, a run script, and the TLC outputs (including the violating traces) are in the `tla/` directory of the source bundle. All models take one account position with two conflicting blocks X and Y , $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch and whose report is chosen adversarially subject to the usability checks of Section V-C. The consensus service is the nondeterministic choice of one eligible candidate from any $N - f$ usable reports. TLC explores all interleavings; a `VIEW` identifies validators up to permutation. Runs used TLC (build 2026-10-06) with two workers.

Invariant names. `NoConflict` and `Agreement`: no two conflicting blocks are final (Theorem 2 at the position). `Thm64`, `Thm64a`, `Thm64b`: Theorem 1 for the respective closure, stated without hypothesis (W). `OneFinal`: a checkpoint finalizes at most one block per position. `SameVerdict`: Lemma 7. `UsableReports`: Lemma 9 once every validator

has left the epoch. `WitnessAdmissible`: line 6 of Algorithm 1 at the next closure can verify a correct F claim against an inherited lock on its rival. `Probe*`: reachability probes, expected to be violated, which show that the invariants are not vacuous.

RaiClose: three voting epochs, two closures. Votes are cast only after the voter's predecessor checkpoint is decided, so there is no early voting, late notarization, or overlap finality. Each closure runs `BuildState` with Rules 1–3, inherited records with origins, and matching-origin discharge; the manifest's discharge set is any subset of the dischargeable records that contains those a selected correct vote relied on. Switch `FIXB` selects the definition of G_i . Abstractions, each of which only adds behaviours: a validator's activity in different epochs is carried by separate identities, so a validator may still sign in epoch e while epoch $e + 1$ votes; a validator that observes its NC releases its final vote in the same step; reports are forgotten once their closure is decided.

RaiOverlap: closing epoch, open epoch, one closure. Validators cross their local boundary one by one. After it, and before installing S_1 , a validator may first-vote in epoch 2 under the cross-epoch guard and then also signs a late notarization in epoch 1. A block may finalize before S_1 is known under the overlap exception, or by rechecking against S_1 after installation. Switches: `NCSPLIT` (a validator records N and final-votes only on an NC of q first votes; off, late notarizations count too), `DISCHARGE` (a recovery record is discharged by a conflicting epoch-1 exclusion witness; off, never), `LATEEV` (late notarizations are admissible manifest evidence for discharge; off, they are “never construction input”). Finality decisions are recorded globally rather than per validator: `ovf` for the overlap exception at a validator that has not installed S_1 , `por` for rechecking at one that has. Closure 2 is not built; its acceptance condition for the case of interest is `WitnessAdmissible`.

RaiHandoff: early voting, two closures, changing committee. Epoch 1 is voted and closed by K_1 , epoch 2 by K_2 ; the Byzantine validator sits in both. A member of K_2 may first-vote in epoch 2 before installing S_1 (an early vote); if it is also in K_1 and supported nothing else in epoch 1, that vote comes with a late notarization in epoch 1. Fix B, the NC split, matching-origin discharge, and late votes as evidence are built in. `RULE = FALSE` is the earlier certificate definition: a fast certificate counts all first votes and a validator final-votes on its epoch-2 NC alone. `RULE = TRUE` is the early-vote rule: a fast certificate counts only first votes cast after the signer installed S_1 , and an early final vote is released only by a validator that holds the epoch-1 exclusion witness. A block is final when its certificate can be formed from released signatures and it is valid against S_1 , through the overlap bundle or by rechecking with discharge. The builder of S_2 holds a discharge witness only if a selected correct reporter relied on it; otherwise the adversary decides, which models a withheld signature. Further abstractions: correct reports carry no F tag for an observed certificate (covered by the other two models);

TABLE I
TLC RUNS. TIMES ARE WALL-CLOCK ON TWO CORES.

Configuration	Result	Time
Close_FALSE	Thm64 violated after 19 steps	35 s
Close_FALSE_conflict	NoConflict violated after 23 steps	2 min
Close_TRUE	no error; 651,632 distinct states	3 min
Overlap_fixed	no error; 5,217,510 distinct states	14 min
Overlap_noDischarge	SameVerdict violated after 15 steps	1 min
Overlap_noLateEv	WitnessAdmissible violated after 14 steps	1 min
Overlap_noSplit	UsableReports violated after 9 steps	10 s
Overlap_noSplit_safety	other four hold; 5,485,444 states	14 min
Overlap_probe1, _probe2	probes violated, as intended	< 1 min
Handoff_same_paper	Thm64b violated after 22 steps	11 min
Handoff_disjoint_paper	Thm64b violated after 23 steps	15 min
Handoff_same_rule	no error; 8,347,138 states	15 min
Handoff_disjoint_rule	no error; 3,707,876 states	17 min
Handoff_overlap3_rule	stopped at depth 18, > 9M states, no violation	–
Handoff_Probe*	probes violated, as intended	< 5 min

a claim that conflicts with a surviving record is ignored rather than invalidating the selection, which only adds checkpoints; epoch 3 is not modelled, so safety after S_2 is checked as Theorem 1 for S_2 .

Table I lists every configuration. The `Overlap_noSplit` run shows that counting late notarizations toward an NC damages usability, hence termination, not safety: with the split off the other four invariants hold over the full state space. The `Handoff` runs with one and four shared members were still running when the archive was made and are not reported. No model covers more than one position, a third epoch, restarts, or any N other than 6.

The random Byzantine voter of the benchmark harness does not find the runs of Appendix A. The following scripted cases were derived from the violating traces; $N = 6$, validator 6 is Byzantine, B and B' conflict.

Case 1 (Fix B). Epoch 1: validators 1, 2, 3 vote B' ; 4, 5 vote B ; 6 reports B only; 3's report is not selected. Epoch 2: 1 to 4 first-vote the retained B , nobody assembles the NC. Release the epoch-1 NC for B' after the second close. Expected: the second checkpoint carries B with an origin-2

record; B finalizes; B' never does.

Case 2 (discharge). Before the boundary 1, 2, 6 vote B' . After it 3, 4, 5 vote B in the open epoch and late-notarize it; 6 joins both. Install the checkpoint at 1 and 2 before they see B 's certificate. Expected: every validator finalizes B , and the next closure builds.

Mixed NC. One validator still in the closing epoch receives three first votes and one late notarization for its block. Expected: it does not record N; its report is usable.

Case 3 (early votes). The run of Appendix A. Expected: B has no fast certificate; 3, 4, 5 final-vote B only if they hold the witness, and then the second checkpoint finalizes B . Without the early-vote rule this run ends with both blocks final.

Case 3, new members. Epoch 1: 1, 2, 3 vote B ; 4, 5 vote B' ; 6 reports B' only and withholds its vote for B . Four members who were not in the old committee early-vote B . Same expectation as Case 3.

`RaiClose` with `FIXB = FALSE` prints Case 1 step by step, `RaiOverlap` with `DISCHARGE = FALSE` prints Case 2, and `RaiHandoff` with `RULE = FALSE` prints both Case 3 runs. In the prototype, Cases 1, 2, Mixed NC, and 3 exist as unit tests of the construction and of the voting rules; a live harness with a scripted Byzantine validator that withholds named votes was not built.

B. Departures from the protocol

First, the manifest commits to signed votes; blocks and ancestry are placed by the report items themselves, and admission witnesses are not committed separately. Second, the consensus instantiation's own votes, the close election's first, final, and timeout votes, are not persisted: durability of the account path, of reports, and of overlap evidence is RAI's, durability of the agreement service is the instantiation's. Third, retained vote records keep a fixed cap of four epochs, since the reports of the next three epochs may still cite an epoch's late finality, and beyond it only the support that may still discharge a surviving recovery record of its origin at its position; only the handoff evidence, an epoch's frozen reports and their reconstructions, is released by acknowledgement, once members of the successor committee carrying $N - f$ of its weight have installed the checkpoint. Fourth, a first vote names its base by one signed bit, settled or early, rather than by the digest of its signer's last installed checkpoint; S_{e-1} is unique, so the bit carries the same information. Fifth, the epoch origin T_0 is not persisted, so the harness cannot restart a validator within a run; the restart path, including the reload of first votes, final votes, late notarizations, frozen reports, and overlap evidence, is exercised by unit tests only.

C. Defects found by the benchmark matrix

Several defects were found and repaired while measuring; none is a failure of the two services. (1) A validator records, for every block it votes for, the parent that block names, so that its residual records place the block; the record was dropped when the slot state of the older of two consecutive epochs that

both voted the block was discarded. The newer report then committed a residual record with a zero parent, BuildState failed at every leader that selected it, and two variants never closed an epoch. The record now outlives every slot state that votes the block, and a report naming a block without its parent is malformed and never selected. (2) A block whose first votes split across an epoch boundary, three cast in the epoch left and two in the next, was retained as a recovery lock and never finalized; every unfinalized instance of the epoch left is now re-voted in the new epoch, and the generator implements the owner policy for the positions that remain. (3) Manifest chunk requests were flooded to every representative and the inbound queues overflowed; requests now go to the proposal’s sender. (4) Re-voting every unfinalized instance filled the election container to its former cap of 5,000, after which no new block was activated on any validator; carried instances whose lock cannot continue are discarded at installation and the cap was raised. (5) Committee derivation deduplicated the epoch’s finalized blocks quadratically, durable vote records shared the ledger’s LMDB writer, and the predecessor gate was opened only after the committee bookkeeping, which together lengthened every wait at the gate. (6) Late notarization first shared a vote generator with the consensus instantiation’s notarization votes and opened an instance of the closing epoch at its receivers for each late vote. (7) After the repairs of Section VII, the manifest’s entries grew to 72 bytes with the late and settled voter bit sets, a full manifest reply overflowed its 16-bit length field, and validators whose own manifest differed from the leader’s silently refused every proposal; the chunk size was reduced. (8) A released epoch was recreated by a peer’s repeated report and re-verified on every tick under the election container’s lock, which stalled vote application for about a second after each installation; the exchange now remembers released epochs. (9) Copies of a proposal that leaders repeat every 200 ms were re-checked in full after the epoch was decided while they held the report-exchange lock, so inbound publishes were dropped and the p95 of one variant swung between 0.4 and 2.9 s between runs; a queued copy now returns once the value is held or the epoch decided.

D. What late notarization buys

On the build of October 7, 2026, before the repairs of Section VII, the nine-variant matrix was run twice on the same day: with the core prefix-wide overlap rule alone and with late notarization added. Without late notarization a block published during a close has no closing-epoch exclusion witness, so every such block waited for the checkpoint: 1,900 to 7,100 instances per boundary, client confirmations stopped for one to three seconds at each close, p95 was 1.3 to 2.4 s without forks and 2.1 to 8.2 s with forks, and 10%-fork goodput was 15 to 25% lower. With late notarization p95 fell four- to eightfold in every variant (no forks: 305, 176, and 186 ms for the all-online, one-offline, and one-Byzantine variants) while medians stayed within 10 ms. Late notarization keeps the core rule and its proof, costs one more signature per block and committee member during each close, and needs no members common

to the two committees. On that build the fork positions whose first votes split three against three stayed retained for the whole run (about 2,100 positions at 5% forks and 4,400 at 10%); the matching-origin discharge and the continuation of every retained block, added with the repairs, are what resolve them in the matrix of the main text.

E. Committee change

The matrix of the main text uses six equal-weight representatives whose committee is the same in every epoch. A second committee model of the prototype keeps the same six members but recomputes their weights from the finalized state at each checkpoint within a bound of $\pm 10\%$ of the equal share and discounts the heaviest members in every threshold; with a workload that moves 5% of one representative’s balance to a fresh holder delegating to the next representative every epoch, the three committees of a run differ from each other, every validator derived the same committee digests from the same checkpoints, and the no-fork variants with all validators online and with one offline settled with latencies close to the equal-weight runs. This exercises committee derivation from the checkpoint state and the per-epoch weighting of certificates, but not a change of identities, which the harness does not yet generate.

F. Reproduction

The prototype, the harness, and the matrix records are in the artifact. One variant is reproduced with

```
python3 tools/rai/run_gate_b.py <dir>
-wait-quiet 600
-fork-percentage 10 -offline 1 -timeout
150
-settle-timeout 90
```

after building the node and the generator in release mode with the `rai_protocol` feature. The evaluation table is regenerated with `tools/eval_table.py`; the models are checked with `tla/run.sh <module> <config>`.